

Appendix: Full $\mathfrak{so}(8)$ Closure Artifact Bundle

HQIV-LEAN

April 29, 2026

Scope

This standalone appendix is intentionally minimal and contains only the symbolic proof-object pipeline and artifacts (exact rational certificates). It omits the full Lean file dump.

Exact Symbolic Certificate (New)

Generator script

Run:

```
PYTHONPATH= /Repos/HQIV python3 scripts/generate_symbolic_lie_closure_certificate.py
```

This script computes a rank-28 closure basis and all bracket coefficients exactly as rational numbers, and writes:

```
artifacts/so8_symbolic_certificate.json
```

scripts/generate_symbolic_lie_closure_certificate.py

```
#!/usr/bin/env python3
"""
Generate an exact (symbolic/rational) Lie-closure certificate for  $\mathfrak{so}(8)$ 
from the HQIV seed generators  $\mathfrak{g}_2 \setminus \{\Delta\}$ , without modifying existing
floating-point certificate files.

Output:
- JSON certificate with:
  * exact basis (packed 28-vectors over  $\mathbb{Q}$ )
  * exact structure coefficients  $c_{\{ij\}}^k$  over  $\mathbb{Q}$ 
  * denominator statistics

Usage:
cd ~/Repos/HQIV_LEAN
PYTHONPATH=~/Repos/HQIV python3 scripts/generate_symbolic_lie_closure_certificate.py
PYTHONPATH=~/Repos/HQIV python3 scripts/generate_symbolic_lie_closure_certificate.py \
--output artifacts/so8_symbolic_certificate.json
"""

from __future__ import annotations

import argparse
import json
import sys
from pathlib import Path
from typing import Iterable
```

```

import sympy as sp

_REPO_LEAN = Path(__file__).resolve().parent.parent
_REPO_HQIV = _REPO_LEAN.parent / "HQIV"
if _REPO_HQIV.exists():
    sys.path.insert(0, str(_REPO_HQIV))

def _to_sympy_matrix(m) -> sp.Matrix:
    """Convert numpy-like matrix with numeric entries to exact sympy Matrix."""
    rows = []
    for i in range(8):
        row = []
        for j in range(8):
            # Current HQVM seed entries are integral / sign values.
            row.append(sp.Rational(int(round(float(m[i][j]))), 1))
        rows.append(row)
    return sp.Matrix(rows)

def _pack_antisym(m: sp.Matrix) -> sp.Matrix:
    """Pack 8x8 antisymmetric matrix to 28x1 vector (upper triangle i<j)."""
    return sp.Matrix([m[i, j] for i in range(8) for j in range(i + 1, 8)])

def _is_zero_matrix(m: sp.Matrix) -> bool:
    return all(v == 0 for v in m)

def _commutator(a: sp.Matrix, b: sp.Matrix) -> sp.Matrix:
    return a * b - b * a

def _rank_of_columns(cols: Iterable[sp.Matrix]) -> int:
    cols = list(cols)
    if not cols:
        return 0
    return sp.Matrix.hstack(*cols).rank()

def build_exact_basis(max_iter: int = 80) -> list[sp.Matrix]:
    """Build a rank-28 Lie-closure basis over Q from g2 {Delta}."""
    from HQVM.matrices import OctonionHQIVAlgebra

    alg = OctonionHQIVAlgebra(verbose=False)
    seed = [_to_sympy_matrix(g) for g in (alg.g2_basis + [alg.Delta])]
    basis: list[sp.Matrix] = []
    basis_cols: list[sp.Matrix] = []

    # Keep seed order, add only rank-increasing directions.
    for m in seed:
        col = _pack_antisym(m)
        if _rank_of_columns(basis_cols + [col]) > len(basis_cols):
            basis.append(m)
            basis_cols.append(col)

    if len(basis_cols) == 28:
        return basis

    # Lie-closure growth by commutators, exact rank tests over Q.
    for _ in range(max_iter):
        grew = False
        n = len(basis)
        for i in range(n):
            for j in range(i + 1, n):
                c = _commutator(basis[i], basis[j])
                if _is_zero_matrix(c):

```

```

        continue
    col = _pack_antisym(c)
    if _rank_of_columns(basis_cols + [col]) > len(basis_cols):
        basis.append(c)
        basis_cols.append(col)
        grew = True
        if len(basis_cols) == 28:
            return basis

    if not grew:
        break

    raise RuntimeError(f"Failed to reach rank 28; got rank {len(basis_cols)}")

def coeff_tensor_exact(basis: list[sp.Matrix]) -> list[list[list[sp.Rational]]]:
    """Compute exact structure constants  $c_{ij}^k$  in the chosen basis."""
    cols = [_pack_antisym(m) for m in basis]
    B = sp.Matrix.hstack(*cols) # 28x28
    if B.rank() != 28:
        raise RuntimeError("Basis matrix is not full rank.")

    coeff: list[list[list[sp.Rational]]] = []
    for i in range(28):
        row = []
        for j in range(28):
            br = _commutator(basis[i], basis[j])
            v = _pack_antisym(br)
            c = B.LUsolve(v) # exact rational vector
            # Ensure exact identity in packed coordinates.
            if B * c != v:
                raise RuntimeError(f"Symbolic solve verification failed at ({i}, {j}).")
            row.append([sp.Rational(c[k]) for k in range(28)])
        coeff.append(row)
    return coeff

def _rat_to_pair(q: sp.Rational) -> tuple[int, int]:
    return int(q.p), int(q.q)

def _matrix_to_pairs(m: sp.Matrix) -> list[list[list[int]]]:
    out = []
    for i in range(m.rows):
        row = []
        for j in range(m.cols):
            p, q = _rat_to_pair(sp.Rational(m[i, j]))
            row.append([p, q])
        out.append(row)
    return out

def main() -> None:
    parser = argparse.ArgumentParser(description="Generate exact symbolic so(8) closure certificate.")
    parser.add_argument(
        "--output",
        default=str(_REPO_LEAN / "artifacts" / "so8_symbolic_certificate.json"),
        help="Output JSON path (default: artifacts/so8_symbolic_certificate.json)",
    )
    parser.add_argument("--max-iter", type=int, default=80, help="Max closure growth iterations.")
    args = parser.parse_args()

    out_path = Path(args.output)
    out_path.parent.mkdir(parents=True, exist_ok=True)

    basis = build_exact_basis(max_iter=args.max_iter)
    coeff = coeff_tensor_exact(basis)

    denoms = [int(c.q) for i in range(28) for j in range(28) for c in coeff[i][j]]

```



```

    0,
    1
]
],
[
    [
        0,
        1
    ]
],
[
    [
        0,
        1
    ]
],
[

```

Toy Model Certificate: $\mathfrak{so}(3) + \Delta_4 \rightarrow \mathfrak{so}(4)$

This compact pedagogical artifact mirrors the exact-rational certificate pipeline in the smallest nontrivial closure setting.

scripts/generate_symbolic_so4_certificate.py

```

#!/usr/bin/env python3
"""
Generate an exact rational toy-model certificate for so(4) closure from
so(3) + Delta4, where Delta4 = J14.

Output:
    artifacts/so4_symbolic_certificate.json
"""

from __future__ import annotations

import json
from pathlib import Path

import sympy as sp

def j(a: int, b: int, n: int = 4) -> sp.Matrix:
    """Standard antisymmetric generator J_ab (1-indexed)."""
    m = sp.zeros(n, n)
    m[a - 1, b - 1] = 1
    m[b - 1, a - 1] = -1
    return m

def pack_antisym(m: sp.Matrix) -> sp.Matrix:
    return sp.Matrix([m[i, k] for i in range(4) for k in range(i + 1, 4)])

def rat_pair(x: sp.Rational) -> list[int]:
    q = sp.Rational(x)
    return [int(q.p), int(q.q)]

def main() -> None:
    out = Path(__file__).resolve().parent.parent / "artifacts" / "so4_symbolic_certificate.json"
    out.parent.mkdir(parents=True, exist_ok=True)

```

```

basis = [j(1, 2), j(1, 3), j(1, 4), j(2, 3), j(2, 4), j(3, 4)]
basis_names = ["J12", "J13", "J14", "J23", "J24", "J34"]
B = sp.Matrix.hstack(*[pack_antisym(x) for x in basis]) # 6x6

# Seed: so(3) on first 3 coords plus Delta4 = J14
seed = [j(1, 2), j(1, 3), j(2, 3), j(1, 4)]
seed_rank = sp.Matrix.hstack(*[pack_antisym(x) for x in seed]).rank()

coeff = []
for i in range(6):
    row = []
    for k in range(6):
        br = basis[i] * basis[k] - basis[k] * basis[i]
        v = pack_antisym(br)
        c = B.LUsolve(v)
        row.append([rat_pair(c[t]) for t in range(6)])
    coeff.append(row)

payload = {
    "description": "Exact rational toy certificate for so(3)+Delta4 -> so(4)",
    "basis_names": basis_names,
    "seed_names": ["J12", "J13", "J23", "Delta4=J14"],
    "seed_linear_rank": int(seed_rank),
    "so4_dimension": 6,
    "witness_brackets": {
        "[J12,J14]": "J24",
        "[J13,J14]": "J34",
    },
    "structure_coeff_q": coeff,
}

with open(out, "w", encoding="utf-8") as f:
    json.dump(payload, f, indent=2)

print(f"Wrote {out}")

if __name__ == "__main__":
    main()

```

artifacts/so4_symbolic_certificate.json

```

{
  "description": "Exact rational toy certificate for so(3)+Delta4 -> so(4)",
  "basis_names": [
    "J12",
    "J13",
    "J14",
    "J23",
    "J24",
    "J34"
  ],
  "seed_names": [
    "J12",
    "J13",
    "J23",
    "Delta4=J14"
  ],
  "seed_linear_rank": 4,
  "so4_dimension": 6,
  "witness_brackets": {
    "[J12,J14]": "J24",
    "[J13,J14]": "J34"
  },
  "structure_coeff_q": [
    [

```



```
],  
[  
  -1,  
  1  
],  
[  
  0,  
  1  
]  
],  
[  
  [  
    0,  
    1  
  ],  
  [  
    1,  
    1  
  ],  
  [  
    0,  
    1  
  ],  
  [  
    0,  
    1  
  ],  
],  
[
```